Лабораторная работа: «Геометрия суперпозиции и Sparse Autoencoders в Transformer-моделях»

Цель работы: Исследовать феномен геометрической суперпозиции признаков в пространствах высокой размерности, понять причины возникновения анизотропии в трансформерах, реализовать и починить Top-K Sparse Autoencoder (SAE) и провести сравнительный анализ контролируемости генерации (Feature Steering vs. Neuron Steering).

Задание 1. Предел концентрации меры и «Паразитное направление» LLM

Теоретический контекст

В пространствах сверхвысокой размерности геометрия подчиняется контринтуитивным законам *концентрации меры*. Если два случайных независимых вектора u, v равномерно распределены на единичной сфере $\mathbb{S}^{d-1}$ (что эквивалентно $x, y \sim \mathcal{N}(0, I_d)$ с последующей L_2-нормализацией), то их скалярное произведение (косинусное сходство) сходится по распределению к нормальному:

$$\langle u, v \rangle \xrightarrow{d} \mathcal{N}\left(0, \frac{1}{d}\right) \quad \text{при } d \to \infty$$

Это означает, что почти любые два случайных вектора в высокой размерности почти ортогональны (разброс $\sigma = 1/\sqrt{d}$).

Однако в реальных больших языковых моделях (LLM) это свойство катастрофически нарушается из-за проблемы дегенерации представлений (*Representation Degeneration Problem*): эмбеддинги сжимаются в узкий конус (анизотропия), а средний косинус между независимыми словами становится строго положительным.

Задачи:

1. **Синтетический тест:** концентрация меры на сфере $\mathbb{S}^{d-1}$

- Напишите векторизованную функцию, которая для заданной размерности d:
  - Генерирует две независимые матрицы $X, Y \in \mathbb{R}^{M \times d}$ из стандартного нормального распределения $\mathcal{N}(0, 1)$ ($M = 20\,000$ пар).
  - Нормализует каждую строку на единичную L_2-норму: $u_i = X_i / \|X_i\|_2,\ v_i = Y_i / \|Y_i\|_2$.
  - Вычисляет построчные скалярные произведения: $s_i = \langle u_i, v_i \rangle$.
- Проведите эксперимент для размерностей $d \in \{16, 64, 256, 512, 4096\}$.
- Для каждого d:
  - Постройте гистограмму распределения s_i и наложите поверх график плотности теоретического распределения $\mathcal{N}(0, 1/d)$.
  - Сведите в итоговую таблицу: теоретическую дисперсию $\sigma^2_{\text{теор}} = 1/d$, выборочную дисперсию $\hat{\sigma}^2$, среднее $\hat{\mu}$ и выборочный эксцесс (kurtosis).

2. **Эксперимент с эмбеддингами Pythia-70m:** «Паразитный конус»
   - Используя библиотеку `transformers`, загрузите веса W матрицы входных эмбеддингов `model.gpt_neox.embed_in.weight` модели `EleutherAI/pythia-70m`. Размер словаря токенизатора модели составляет $50\,277$, а размер матрицы W — $50\,304$ (дополнен нулями/шумом для кратности 64 под Tensor Cores). Обрежьте матрицу до реального размера словаря: `W = W[:50277]`.
   - Нормализуйте все вектор-строки на единичную сферу.
   - Сэмплируйте $10\,000$ случайных пар токенов и постройте гистограмму $\langle u, v \rangle$. Сравните ее с теоретической кривой $\mathcal{N}(0, 1/512)$. Вас ждет парадокс: распределение не совпадет с теорией $\mathcal{N}(0, 1/512)$ (должен получиться колокол, сдвинутый вправо: среднее значение $\mu \gg 0$, обычно $\mu \in [0.15, 0.35]$. Эмбеддинги смотрят «в одну сторону», занимая узкий телесный угол).
   - Объясните математически и физически, почему в авторегрессионных языковых моделях возникает общий паразитный конус. Проконсультируйтесь с ИИ как его исправить и добейтесь совпадения с теорией.

Задание 2. Toy Model of Superposition: интерференция и геометрия упаковки

1. Контекст и теоретическая модель

Мы исследуем явление **суперпозиции** (способности нейросетей упаковывать большее число независимых признаков M, чем доступно измерений d, $M > d$) при условии высокой разреженности (sparsity) признаков.

- **Вектор признаков:** $f \in \mathbb{R}^M$, где компоненты независимы:

  $$f_i = r_i \cdot b_i, \quad b_i \sim \text{Bernoulli}(p), \quad r_i \sim \text{Uniform}(0, 1)$$

  Здесь $p \in (0, 1)$ — вероятность активации признака (уровень разреженности $S = 1 - p$).
- **Сжатие в скрытый слой:** $x = Wf$, где $W \in \mathbb{R}^{d \times M}$, а столбцы $w_i \in \mathbb{S}^{d-1}$ ($\|w_i\|_2 = 1$) являются направлениями признаков.
- **Восстановление (декодирование):**

  $$\hat{f} = \text{ReLU}(W^T x - b) = \text{ReLU}\left(W^T W f - b\right)$$

  где $b \in \mathbb{R}^M$ — вектор порогов отсечения шума (или скаляр $b \cdot \mathbf{1}$).

Раскладывая i-ю компоненту выхода, получаем:

$$\hat{f}_i = \text{ReLU}\left(f_i + \sum_{j \neq i} (w_i^T w_j) f_j - b_i\right)$$

Второе слагаемое — это **интерференционный шум** от других активных признаков.

---

2. Задачи

Зафиксируйте размерности: $d = 10$, $M = 40$ (коэффициент сжатия $4\times$). Исследуйте диапазон $p \in [0.01, 0.5]$ (возьмите 15–20 точек).

Задача 2.1. Генерация случайной схемы (Random Projection)

1. Сгенерируйте матрицу $W_{rand} \in \mathbb{R}^{d \times M}$, элементы которой взяты из $\mathcal{N}(0, 1)$, и нормируйте каждый столбец на единичную ℓ_2-норму.
2. Сгенерируйте тестовый батч векторов $F \in \mathbb{R}^{M \times N}$ ($N \geq 20\,000$ примеров для статистической стабильности).

Задача 2.2. Поиск оптимального порога фильтрации шума Для каждого значения p: 1. Объясните теоретически: почему при $b = 0$ модель с ReLU выдаёт неприемлемо большую ошибку даже при ультра-разреженных данных ($p = 0.01$)? 2. Найдите оптимальный скалярный порог $b^*(p)$, минимизирующий ошибку восстановления:

$$b^*(p) = \arg\min_b \mathbb{E}[\|f - \text{ReLU}(W^T W f - b)\|_2^2]$$

(Подсказка: используйте одномерную оптимизацию, например `scipy.optimize.minimize_sc` *или сеточный поиск Grid Search по диапазону* $[0.0, 1.0]$*).* 3. Постройте график зависимости оптимального b^* от p. Как меняется оптимальный порог с ростом вероятности активности признаков и почему?

Задача 2.3. Фазовая диаграмма и анализ «коллапса»

1. Постройте графики качества восстановления от вероятности активации p:
   - **Осторожно (Ловушка метрики):** Если вы построите просто сырой $MSE(p)$, график покажет тривиальный монотонный рост, так как сама дисперсия входных данных $\mathbb{E}[\|f\|^2] = \frac{Mp}{3}$ растет с ростом p!
   - **Правильный шаг:** Вычислите **Нормированный MSE (NMSE)**:

$$\text{NMSE}(p) = \frac{\mathbb{E}[\|f - \hat{f}\|_2^2]}{\mathbb{E}[\|f\|_2^2]}$$

   Сравните качество вашей модели с **тривиальным baseline** (модель $\hat{f}_{zero} = 0$, для которой NMSE $= 1.0$).
2. Найдите точку коллапса p^*, в которой NMSE ≥ 0.95 (модель перестает превосходить тривиальное зануление).

Задача 2.4. «Ловушка случайной матрицы» vs Настоящая суперпозиция

- **Ловушка:** В задаче 2.1 матрица W была зафиксирована случайно. При случайном расположении 40 векторов в $\mathbb{R}^{10}$ взаимная неколлинеарность ограничена, интерференция случайна и разрушает сигнал уже при малых p.

- **Исправление:** Обучите W и вектор смещений $b \in \mathbb{R}^M$ градиентным спуском (PyTorch, Adam, ~2000–5000 шагов) под каждое конкретное значение $p \in \{0.01, 0.05, 0.1, 0.2\}$, минимизируя $\mathcal{L} = \frac{1}{N}\|f - \text{ReLU}(W^T W f - b)\|_2^2$. Столбцы W после каждого шага проецируйте обратно на сферу $\|w_i\|_2 \leq 1$.
- **Сравнение:** Постройте кривую NMSE обученной модели поверх кривой случайной модели.
- **Геометрический анализ:** Для обученной матрицы W визуализируйте матрицу Грама $G = W^T W \in \mathbb{R}^{M \times M}$. Увидите ли вы структурные паттерны (ортогональные подпространства, антиподальные пары $w_i \approx -w_j$)?

Задание 3. Взлом и починка Top-K Sparse Autoencoder на активациях Pythia-70m

Введение и контекст

Разреженные автоэнкодеры (Sparse Autoencoders, SAE) позволяют разложить скрытые репрезентации трансформеров на интерпретируемые моносемантичные признаки. В отличие от классических L1-SAE, где разреженность достигается штрафом за регуляризацию, в архитектуре **Top-K SAE** (OpenAI / Gao et al., 2024) разреженность задается жестко: ровно K самых активных нейронов пропускаются дальше, а остальные зануляются.

Ниже приведен прототип автоэнкодера, перехватывающего residual stream 3-го слоя модели `EleutherAI/pythia-70m` ($d = 512$, словарь $D = 2048$, $K = 32$). Код содержит **три критические архитектурные ошибки/ловушки**, из-за которых модель коллапсирует: Explained Variance не поднимается выше 50-60%, а более 40% признаков становятся «мертвыми».

```
import torch
import torch.nn as nn

class BrokenTopKSAE(nn.Module):
    def __init__(self, d_model=512, d_sae=2048, k=32):
        super().__init__()
        self.k = k
        self.W_enc = nn.Parameter(torch.randn(d_model, d_sae) * 0.02)
        self.b_enc = nn.Parameter(torch.zeros(d_sae))
        self.W_dec = nn.Parameter(torch.randn(d_sae, d_model) * 0.02)
        self.b_dec = nn.Parameter(torch.zeros(d_model))

    def encode(self, x):
        pre_acts = x @ self.W_enc + self.b_enc
        val, idx = torch.topk(pre_acts, self.k, dim=-1)
        acts = torch.zeros_like(pre_acts).scatter_(-1, idx, val)
        return acts

    def decode(self, acts):
```

```
        return acts @ self.W_dec + self.b_dec

    def forward(self, x):
        acts = self.encode(x)
        x_hat = self.decode(acts)
        return x_hat, acts
```

Задачи:

1. Запрограммировать (оптимально!) и запустить обучение на CPU (1000 шагов, batch_size=64, занимает ~2-3 минуты на Core i5/Ryzen 5).
2. **Замерить ключевые метрики за последние 500 шагов обучения:**
   - **Explained Variance (Объясненная дисперсия):**

$$\text{EV} = 1 - \frac{\text{Var}(x - \hat{x})}{\text{Var}(x)}$$

     (Вычисляется поэлементно или по всему батчу; дисперсия берется вдоль размерности признаков).
   - **Dead Latents Ratio (Доля мертвых признаков):** Доля латентов (из 2048), которые **ни разу** не вошли в Top-K на протяжении последних 500 шагов (т.е. за $500 \times 64 = 32\,000$ обработанных токенов).
3. Убедится, что исходный автоэнкодер поломан и реализовать исправление класса TopKSAE. **Правильные значения метрик:**
   - Explained Variance $\geq 80\%$.
   - Dead Latents Ratio $\leq 5\%$ (не более ~100 мертвых нейронов из 2048).

Задание 4. Механистическая интерпретируемость: Feature Steering против Neuron Steering

Контекст

В рамках гипотезы суперпозиции модель упаковывает большее число понятий, чем количество доступных измерений в скрытом пространстве. Это приводит к тому, что: 1. Базисные направления (отдельные нейроны) являются **полисемантичными** (кодируют смесь независимых концептов или служат для подавления артефактов оптимизации). 2. Направления латентов словаря Sparse Autoencoder (SAE) являются **моносемантичными** (выделяют изолированный концепт).

Задачи:

1. Подготовка и калибровка шкалы возмущения Чтобы эксперимент был математически строгим, мы не можем использовать произвольные числа (вроде $+50.0$). * Измерьте среднюю евклидову норму вектора активаций residual stream $\mathbb{E}[\|x\|_2]$ на выбранном слое L для нейтральных текстов. * Для всех последующих экспериментов используйте **аддитивное воздействие одинаковой мощности**:

$$x_{\text{steered}} = x + \alpha \cdot v$$

где v — единичный вектор направления ($\|v\|_2 = 1$), а α — варьируемая амплитуда (задайте сетку $\alpha \in [0.1 \cdot \|x\|, 1.5 \cdot \|x\|]$).

2. Эксперимент А: Воздействие на базисный нейрон (Neuron Steering)

- Выберите ортонормированный базисный вектор e_i (где i — индекс нейрона residual stream, например, $i = 42$ или $i = 128$).
- Примените интервенцию: $x \leftarrow x + \alpha \cdot e_i$.
- Сгенерируйте продолжение нейтрального промпта: `"In the middle of the room there was a large table with"` (25 новых токенов).

  Внимание: Если вы попытаетесь вручную подменять тензоры до вызова `model.generate()`, генерация сорвется или изменится только первый токен. **Что делать:** Используйте PyTorch Forward Hook (`register_forward_hook`). Помните, что во время авторегрессии на первом шаге размерность тензора составляет `[batch, seq_len, d_model]`, а на последующих шагах (из-за KV-кэша) — `[batch, 1, d_model]`. Ваша функция хука должна корректно бродкаститься по этим размерностям.

Вопросы к анализу: - Удается ли интерпретировать тему сгенерированного текста? - Проявляется ли полисемантичность (хаотичное переключение между не связанными темами) или модель просто теряет способность формировать синтаксис (дегенерация)?

3. Эксперимент Б: Поиск моносемантического вектора SAE

- Вам необходимо найти латент j, кодирующий конкретную узкую фичу (например, «программный синтаксис / скобки» или «кавычки / прямая речь»).
- Прогоните проверочный датасет из 20–30 предложений через энкодер SAE:

$$f(x) = \text{ReLU}(W_{\text{enc}}(x - b_{\text{dec}}) + b_{\text{enc}})$$

- Найдите латент j, для которого соблюдается условие специфичности:
  1. $\mathbb{E}[f_j(x_{\text{target}})] \gg 0$ на целевых токенах (например, на токенах `(`, `{`, `[`).
  2. $f_j(x_{\text{neutral}}) = 0$ на 99% остальных токенов.

  Осторожно: Если вы выберете латент просто по максимальной активации на одном предложении, велик риск выбрать «латент пунктуации вообще» или артефактный латент плотности текста. Обязательно визуализируйте топ-5 токенов с максимальной активацией выбранного латента j, чтобы убедиться в его моносемантичности перед переходом к стирингу.

4. Эксперимент В: Направленное воздействие на вектор фичи (Feature Steering)

- Возьмите соответствующий вектор столбца декодера SAE: $d_j = W_{\text{dec}}[:, j]$. Нормализуйте его: $v_{\text{SAE}} = \frac{d_j}{\|d_j\|_2}$.
- Примените интервенцию: $x \leftarrow x + \alpha \cdot v_{\text{SAE}}$ на том же слое L с тем же промптом и тем же набором значений силы воздействия α, что и в Эксперименте А.
- Сгенерируйте продолжение текста.

Контрольные задания и отчетность:

1. **Качественный анализ:** Приведите сравнительную таблицу текстов, сгенерированных:
   - Базовой моделью ($\alpha = 0$).
   - Моделью с Neuron Steering ($v = e_i$) для трех значений α (слабое, среднее, сильное).
   - Моделью с Feature Steering ($v = v_{\text{SAE}}$) для тех же трех значений α. Опишите, как меняется семантика текста в обоих случаях.
2. **Количественный анализ (Перплексия vs Моносемантичность):**
   - Постройте график зависимости Language Modeling Loss (Cross-Entropy на нейтральном корпусе) от величины α для нейрона и для фичи SAE.
   - *Объясните результат:* Почему при одинаковой норме добавленного вектора SAE-вектор дольше сохраняет синтаксическую связность текста (низкую перплексию), чем базисный нейрон?
3. **Фазовый переход в бред:**
   - При каком пороговом значении α направленное воздействие SAE перестает вносить концепт и начинает разрушать связность (вызывать зацикливание, повторение токенов или несвязные символы)? С чем это связано с точки зрения геометрии скрытого пространства?